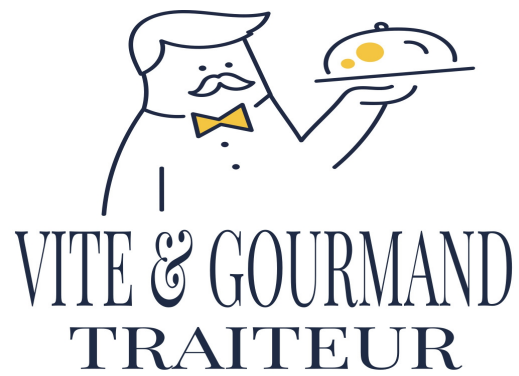




VITE & GOURMAND

Explication du MCD

Projet ECF

Titre professionnel

Développeur Web et Web Mobile

Yoann LAMOUILLE

2026

Explication de la conception du MCD

Vite & Gourmand

Ce document présente les principaux choix réalisés lors de la conception du modèle conceptuel de données (**MCD**) de l'application **Vite & Gourmand**. L'objectif du modèle est de représenter les données nécessaires aux fonctionnalités de l'application tout en conservant un historique fiable, en limitant les duplications inutiles et en traduisant les principales règles métier.

1 Principes généraux de conception

Séparation des responsabilités : Les données ont été réparties dans plusieurs entités afin d'éviter de regrouper des informations de natures différentes dans une seule table. Par exemple, les utilisateurs, les commandes, les menus, les plats, les avis et les statuts disposent chacun de leur propre entité.

Nommage : Les noms des tables et des attributs utilisent principalement la convention `snake_case`. Les identifiants sont nommés avec le préfixe `id_`, par exemple `id_utilisateur`, `id_commande` ou `id_menu`. Cette convention facilite la lecture du modèle et sa correspondance avec la base MySQL.

1.1 Gestion des cardinalités et des clés étrangères

Les cardinalités définies dans le **MCD** déterminent la manière dont les relations sont ensuite traduites dans la base de données relationnelle.

Dans une relation de type **one-to-many (1,N)**, la clé primaire de l'entité située du côté « un » devient une clé étrangère dans la table située du côté « plusieurs ». Par exemple, un utilisateur peut passer plusieurs commandes, tandis qu'une commande est associée à un seul utilisateur. La table **commande** contient donc la clé étrangère **utilisateur_id**, qui référence **utilisateur.id**.

Dans une relation de type **many-to-many (N,N)**, une table d'association est créée entre les deux entités. Elle contient les clés étrangères correspondant aux clés primaires des deux tables associées. C'est notamment le cas de **menu_plat**, qui permet d'associer plusieurs plats à plusieurs menus, et de **plat_allergene**, qui permet d'associer plusieurs allergènes à plusieurs plats.

Le projet ne nécessite pas de relation de type **one-to-one (1,1)**. Ce type de relation pourrait par exemple être utilisé dans une entreprise si chaque employé disposait d'une seule place de parking et que chaque place était attribuée à un seul employé. Dans le modèle relationnel, une clé étrangère associée à une contrainte d'unicité permettrait de garantir cette relation.

Ces règles permettent de traduire les cardinalités du MCD dans la base de données relationnelle et d'assurer la cohérence des relations entre les différentes tables.

2 Utilisateurs, rôles et réinitialisation du mot de passe

Utilisateur et Role : l'entité Utilisateur regroupe les comptes de l'application. Le rôle n'est pas stocké directement sous forme de texte dans chaque utilisateur : une entité Role distincte permet de centraliser les rôles Client, Employé et Admin. Un rôle peut être attribué à plusieurs utilisateurs, tandis qu'un utilisateur possède un seul rôle.

Réinitialisation_mot_de_passe : les demandes de réinitialisation sont séparées de l'entité Utilisateur. Cette entité contient notamment le hash du jeton et ses dates de création et d'expiration. Ce choix évite de stocker ces données temporaires directement dans le compte utilisateur et permet de gérer la durée de validité d'un lien de réinitialisation.

3 Commandes et conservation de l'historique

Duplication volontaire des informations du client : j'ai fait le choix de dupliquer dans Commande certaines informations du client, notamment le nom, le prénom, l'e-mail et le téléphone, même si elles existent également dans Utilisateur. Cette duplication est volontaire : elle permet de conserver les informations utilisées au moment de la commande si le client modifie ensuite son profil. L'historique d'une commande reste ainsi fidèle à la situation au moment où elle a été passée.

Lien avec Utilisateur : la commande reste néanmoins liée au compte utilisateur qui l'a effectuée. Ce lien permet d'afficher l'historique des commandes dans l'espace client et de vérifier qu'un utilisateur accède uniquement à ses propres commandes.

Informations propres à la commande : Commande contient les données qui décrivent l'opération réalisée : nombre de personnes, prix unitaire, adresse, date et heure de livraison, frais de livraison, réduction, prix total, prêt de matériel et dates associées. Ces valeurs correspondent à la commande telle qu'elle a été enregistrée.

4 Menus, plats, thèmes, régimes et images

Menu et Plat : les menus et les plats sont séparés car un menu est une offre commerciale composée de plusieurs plats. La relation entre ces deux entités est gérée par l'association Menu_plat. Cette table d'association permet de représenter une relation many-ton-many sans dupliquer les informations d'un plat.

Theme et Regime : les thèmes et les régimes ont été modélisés comme des entités distinctes plutôt que comme de simples textes dans Menu. Cela évite les variations d'écriture, facilite la cohérence des données et permet de réutiliser une même valeur pour plusieurs menus.

Image : les images sont séparées des menus afin de pouvoir associer plusieurs images à un même menu. L'attribut ordre_affichage permet de maîtriser leur ordre de présentation et texte_alternatif permet de fournir une description de l'image.

5 Allergènes

Plat, Allergene et Plat_allergene : un plat peut contenir plusieurs allergènes et un même allergène peut être présent dans plusieurs plats. Cette relation plusieurs-à-plusieurs est donc représentée par l'association Plat_allergene. Les allergènes ne sont pas répétés sous forme de texte dans chaque plat, ce qui facilite leur gestion et réduit les incohérences.

6 Suivi des commandes et statuts

Statut_commande : les statuts sont regroupés dans une entité dédiée afin d'utiliser une liste cohérente de valeurs pour l'ensemble des commandes.

Suivi_commande : le suivi est séparé de la commande afin de conserver les changements de statut successifs avec leur date. Une commande peut ainsi posséder plusieurs lignes de suivi. Ce choix permet de conserver l'historique de son évolution au lieu de connaître uniquement son état actuel.

7 Ville de livraison et distance

Ville_commande : la ville et la distance utilisées pour la livraison sont représentées séparément afin de disposer de la distance nécessaire au calcul des frais de livraison. La commande reste liée aux informations de livraison nécessaires à l'application des règles tarifaires.

8 Avis clients

Avis : un avis est lié à la commande ainsi qu'à l'utilisateur qui l'écrit. Cette organisation permet de vérifier que l'avis provient d'un client ayant réellement passé la commande concernée. L'attribut est_valide permet de distinguer un avis en attente de modération d'un avis validé et affichable dans l'application.

9 Restaurant, horaires et messages de contact

Restaurant et Horaire : les horaires sont séparés du restaurant afin de stocker les horaires de chaque jour sans multiplier les colonnes dans l'entité Restaurant. Un restaurant peut ainsi disposer de plusieurs lignes d'horaires.

Message_contact : les demandes envoyées depuis le formulaire de contact disposent d'une entité dédiée contenant les coordonnées fournies par le visiteur, le sujet, le message et la date d'envoi. Elles sont liées au restaurant destinataire sans dépendre de l'existence d'un compte utilisateur.

10 Correspondance avec les principales règles métier

Besoin / règle métier	Choix dans le MCD
Conserver l'historique d'une commande	Informations client et valeurs de la commande conservées dans Commande.
Composer un menu de plusieurs plats	Association Menu_plat entre Menu et Plat.
Gérer plusieurs allergènes par plat	Association Plat_allergene entre Plat et Allergene.
Suivre l'évolution d'une commande	Entités Statut_commande et Suivi_commande avec date de changement.
Calculer les frais de livraison	Ville_commande conserve notamment la distance utilisée par la règle tarifaire.
Permettre et modérer les avis	Avis lié à la commande et à l'utilisateur, avec l'attribut est_valide.
Gérer plusieurs rôles	Entité Role reliée à Utilisateur.
Sécuriser la réinitialisation du mot de passe	Entité Réinitialisation_mot_de_passe avec hash et date d'expiration.

11 Conclusion

Le MCD a été construit à partir des besoins fonctionnels et des règles métier de Vite & Gourmand. Les choix de séparation des entités, de création des associations et de conservation de certaines données historiques visent à obtenir un modèle compréhensible, cohérent avec l'application et adapté à son évolution. Certaines informations sont volontairement dupliquées lorsqu'elles représentent un état historique à conserver, tandis que les données réutilisables sont centralisées dans des entités dédiées afin de limiter les incohérences.